


**Faculty of Engineering and Technology**

**Electrical and Computer Engineering Department**

**ENCS5337, CHIP DESIGN VERIFICATION**

**Project Interrupt Controller**

---

Prepared by:

**Mohammed Obied 1210093**

**Malik Arqoup 1211686**

Instructor: Dr.Ayman Hroub

Sec:1

Date:10/8/2025

# Introduction

The aim of this project is to verify the design of an **Interrupt Controller** and ensure that it operates correctly according to the specified requirements. The interrupt controller is a key component in processing systems, responsible for managing interrupt requests (IRQs) from multiple sources, determining service priority, controlling the enabling or disabling of interrupts through mask registers, and handling acknowledgment (ACK) signals from the processor.

In this work, the **UVM (Universal Verification Methodology)** was used to build a complete verification environment, including all standard testbench components such as the driver, monitor, and scoreboard, to generate test scenarios and analyze the results. The verification process included both directed tests and constrained-random tests to cover all possible scenarios, including normal operation and corner cases.

The overall goal of the project is to ensure that the interrupt controller performs efficiently and reliably under various operating conditions, meeting all design requirements in terms of priority resolution, masking, concurrent interrupt handling, and reset functionality.

# Table of contents

## Contents

Introduction ..... 2
Table of contents ..... 3
Table of figure ..... 4
Design and Implementation Description ..... 5
Pending Register (pending_reg) ..... 5
Mask Register (mask_reg) ..... 5
Active IRQ Logic (active_irqs) ..... 5
Priority Encoder ..... 5
Acknowledge Handling (ACK) ..... 5
Reset Handling (Rstn) ..... 6
Block Diagram Description ..... 7
Detailed verification plan ..... 8
Scope and Goals ..... 8
Methodology ..... 8
Testbench Architecture ..... 9
Test Cases ..... 9
Sign-off Criteria ..... 10
Simulation snapshots for all tests ..... 11
1.Simulation Snapshot – All IRQs Active ..... 11
.2 Simulation Snapshot – All IRQs Inactive ..... 11
3. Simulation Snapshot – Mixed Priority Scenario (IRQ5, IRQ6, late IRQ1, then IRQ7) ..... 12
4.Masking Test – Ignored IRQ1, Served IRQ2 ..... 13
5.Testing Full Interrupt Mask Activation ..... 14
Conclusion ..... 15

# Table of figure

Figure 1 Block Diagram of the Interrupt Controller ..... 7
Figure 2 All IRQ lines enabled code ..... 11
Figure 3 All IRQ lines enabled output ..... 11
Figure 4 All IRQ lines disabled code ..... 12
Figure 5 All IRQ lines disabled output ..... 12
Figure 6 Scenario setup: IRQ5, IRQ6, IRQ7 active → after delay 10, IRQ1 arrives code ..... 13
Figure 7 Scenario setup: IRQ5, IRQ6, IRQ7 active → after delay 10, IRQ1 arrives output ..... 13
Figure 8 IRQ1 masked, only IRQ2 serviced code ..... 13
Figure 9 IRQ1 masked, only IRQ2 serviced output ..... 14
Figure 10All IRQ lines masked code ..... 14
Figure 11All IRQ lines masked output ..... 14

# Design and Implementation Description

The **Interrupt Controller** is designed to ensure that the CPU always services the highest-priority active interrupt while allowing for flexible masking and acknowledgment mechanisms. The design guarantees accurate edge detection, interrupt prioritization, and selective enabling/disabling of interrupt lines according to the given specifications.

**Main Functional Blocks**

## Pending Register (pending_reg)

- Captures all active interrupts based on rising edges of IRQ signals.
- Stores the set of interrupts that are currently awaiting service.
- Updates only when new interrupts occur, ensuring minimal unnecessary changes.

## Mask Register (mask_reg)

- Stores the masking configuration provided by the mask_in input.
- If a mask bit is set to 1, the corresponding interrupt is ignored.
- Enables selective enabling or disabling of specific interrupt lines.

## Active IRQ Logic (active_irqs)

- Combines pending_reg and mask_reg using a bitwise AND with the inverted mask.
- Produces the set of interrupts that are both pending and unmasked.

## Priority Encoder

- Scans active_irqs to find the highest-priority interrupt.
- Priority is fixed: IRQ0 is the highest, and IRQ7 is the lowest.
- Outputs the corresponding ID (IRQ_ID) and asserts IRQ_OUT when a valid interrupt is detected.

## Acknowledge Handling (ACK)

- When ACK is asserted by the CPU, the current highest-priority pending interrupt is cleared from pending_reg.

- If multiple interrupts remain active, the next highest-priority interrupt is selected in the following clock cycle.

## Reset Handling (Rstn)

- On active-low reset, all registers are cleared, IRQ_OUT is de-asserted, and IRQ_ID is reset to zero.

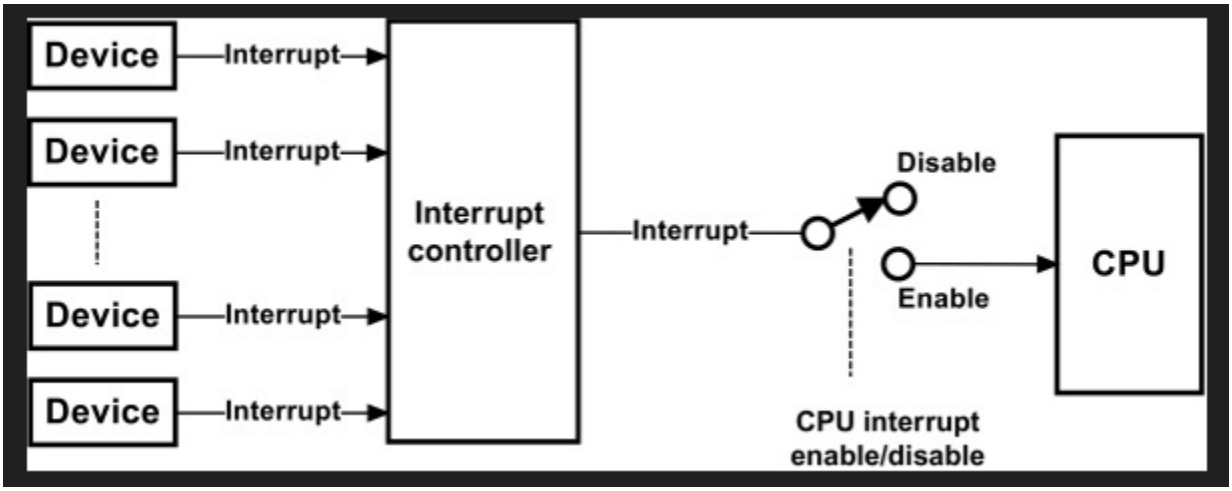



*Figure 1 System-Level Block Diagram*

This diagram illustrates the overall connection between devices, the interrupt controller, and the CPU. Devices send interrupt requests to the controller, which manages them based on priority and masking logic before signaling the CPU. The CPU can globally enable or disable interrupts.

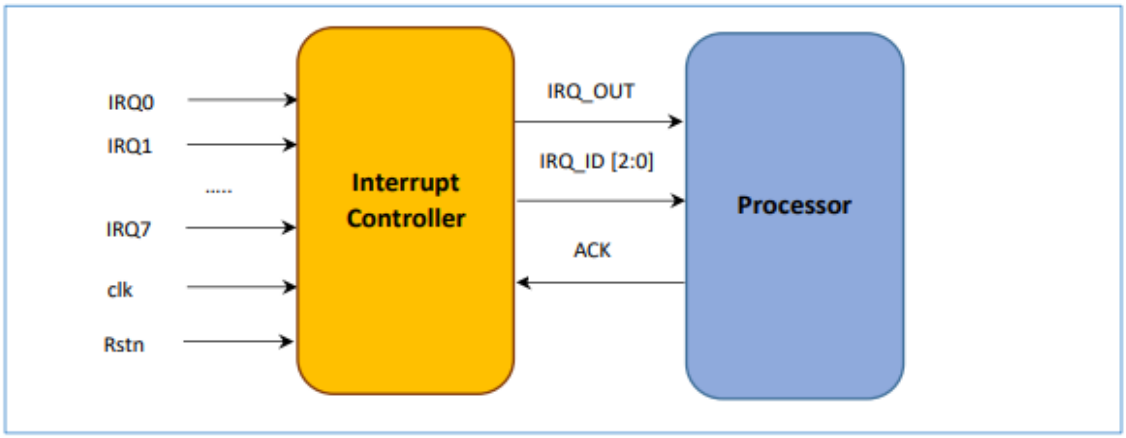



*Figure 2 Internal Block Diagram*

## Block Diagram Description

Figure 1 illustrates the high-level architecture of the Interrupt Controller system. The controller receives up to eight interrupt request signals (IRQ0–IRQ7), along with the system clock (clk) and active-low reset (Rstn). It processes these inputs according to the priority logic, masking configuration, and pending register status. The output signals include IRQ_OUT, which indicates a valid interrupt to the processor, and IRQ_ID, which specifies the ID of the highest-priority active interrupt. The processor responds with an acknowledgment (ACK) to clear the serviced interrupt from the pending list.

# Detailed verification plan

## Scope and Goals

The goal of the verification process is to ensure that the Interrupt Controller meets all functional and performance requirements as per the design specifications. The scope covers:

Handling of all interrupt lines (IRQ0–IRQ7) with correct priority resolution.

Accurate application of masking logic via the mask register.

Correct response to acknowledgment (ACK) signals.

Support for concurrent interrupts and preemption, where higher-priority interrupts can interrupt lower-priority ones.

Edge cases such as when all interrupts are masked, or when a high-priority interrupt arrives during the processing of a lower-priority one.

Verification will confirm that the DUT behaves correctly in normal and stress conditions.

## Methodology

UVM based testbench will be used in the verification. It will have directed testing and constrained-random tests to have as much ground being covered as possible: Priorities of the active interrupts. Masking behaviour with different combinations of IRQs masked/unmasked. ACK, succeeded by behavior so that pending interrupts are cleared properly. Preemptive conditions, to justify good resumption of interrupted lower preference trends. The stress tests will involve enabling different IRQs at the same time and mask toggling over the course of service.

## Testbench Architecture

The intended testbench will comprise the standard UVM:

Driver: Drives IRQ and mask to DUT.

Monitors: Records the outputs of DUT.

Scoreboard: Performs similarity test between DUT outputs and expected outcomes of a golden model.

Main interfaces:

Inputs: clk, rstn, irq[7:0], mask_in[7:0], ack.

Outputs: IRQ_OUT, IRQ_ID.

Sample sets of tests will be built up in order to verify the correct and incorrect functionality, including the edge cases.

## Test Cases

The following test cases will be planned:

Basic Operation: Test proper priority-based servicing of IRQs in regular situations.

Functionality of the mask: make sure that the masked interrupts are ignored whether the coffee is active or not.

Preemption: it is possible to confirm higher-priority IRQs to preempt a lower-priority IRQ, and properly resume.

Reset Behavior: Check Reset Behavior, clearing of pending interrupt and returning to default state.

Concurrent IRQ Activation: Test appropriate behaviour under common occurrences in which two or more IRQs are active concurrently.

## Sign-off Criteria

Verification will be considered complete when:

All planned test cases have been executed and passed.

Functional and code coverage goals (100%) have been met.

No open high-priority bugs remain unresolved.

Verification results have been reviewed and approved by the design team.

# Simulation snapshots for all tests

## 1.Simulation Snapshot – All IRQs Active

All interrupt lines (IRQs) were enabled, and the system serviced all requests in order of priority.

```
    tr = irq_seq_item::type_id::create("tr_all_on");
tr.irq     = 8'hFF;
tr.mask_in = 8'h00;
tr.rstn    = 1'b1;
tr.ack     = 1'b1;
start_item(tr);
finish_item(tr);
```

*Figure 3 All IRQ lines enabled code*

```
UVM_INFO irq_scoreboard(51) @ 15000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=0
UVM_INFO irq_scoreboard(70) @ 25000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=0) == REF(OUT=1,ID=0) ref reg 11111110
UVM_INFO irq_scoreboard(51) @ 25000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=1
UVM_INFO irq_scoreboard(70) @ 35000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=1) == REF(OUT=1,ID=1) ref reg 11111100
UVM_INFO irq_scoreboard(51) @ 35000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=2
UVM_INFO irq_scoreboard(70) @ 45000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=2) == REF(OUT=1,ID=2) ref reg 11111000
UVM_INFO irq_scoreboard(51) @ 45000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=3
UVM_INFO irq_scoreboard(70) @ 55000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=3) == REF(OUT=1,ID=3) ref reg 11110000
UVM_INFO irq_scoreboard(51) @ 55000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=4
UVM_INFO irq_scoreboard(70) @ 65000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=4) == REF(OUT=1,ID=4) ref reg 11100000
UVM_INFO irq_scoreboard(51) @ 65000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=5
UVM_INFO irq_scoreboard(70) @ 75000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=5) == REF(OUT=1,ID=5) ref reg 11000000
UVM_INFO irq_scoreboard(51) @ 75000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=6
UVM_INFO irq_scoreboard(70) @ 85000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=6) == REF(OUT=1,ID=6) ref reg 10000000
UVM_INFO irq_scoreboard(51) @ 85000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=7
UVM_INFO irq_scoreboard(70) @ 95000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=7) == REF(OUT=1,ID=7) ref reg 00000000
```

*Figure 4 All IRQ lines enabled output*

## 2. Simulation Snapshot – All IRQs Inactive

All interrupt lines (IRQs) were disabled, and no interrupt service was performed.

```
// --- Transaction 2: all IRQ lines off ---
tr = irq_seq_item::type_id::create("tr_all_off");
tr.irq     = 8'h00;
tr.mask_in = 8'h00;
tr.rstn    = 1'b1;
tr.ack     = 1'b1;
start_item(tr);
finish_item(tr);
#70;
```

*Figure 5 All IRQ lines disabled code*

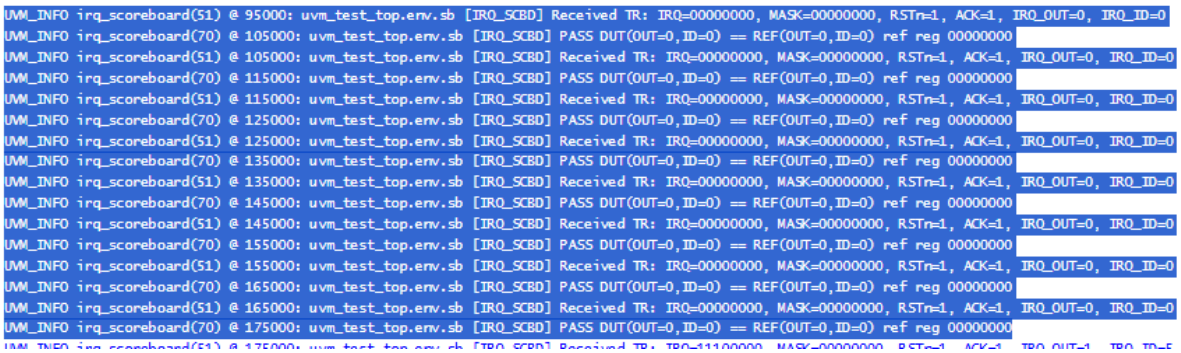



*Figure 6 All IRQ lines disabled output*

## 3. Simulation Snapshot – Mixed Priority Scenario (IRQ5, IRQ6, late IRQ1, then IRQ7)

In this test, interrupt lines 5, 6, and 7 were initially activated, and the system began servicing them in order of priority (IRQ5 followed by IRQ6). While completing the service for IRQ6, and after approximately 10 cycles, IRQ1 arrived, which has a higher priority than IRQ7. The system paused before moving to IRQ7 and serviced IRQ1 first. After completing IRQ1, it resumed and completed servicing IRQ7. The purpose of this scenario is to verify the system's ability to interrupt its current service sequence when a higher-priority interrupt arrives, and then return to handle the remaining pending interrupts.

```
// --- Transaction 1: all IRQ lines high ---
tr = irq_seq_item::type_id::create("tr_all_high");
tr.irq     = 8'b11100000;    // IRQ0..7 all asserted
tr.mask_in = 8'h00;     // no masking
tr.rstn    = 1'b1;      // not in reset
tr.ack     = 1'b1;      // ACK de-asserted
start_item(tr);
finish_item(tr);

// Wait one cycle so the DUT can react
#10;

// --- Transaction 2: all IRQ lines cleared ---
tr = irq_seq_item::type_id::create("tr_all_zero");
tr.irq     = 8'h01;     // no IRQs
tr.mask_in = 8'h00;
tr.rstn    = 1'b1;
tr.ack     = 1'b1;
start_item(tr);
finish_item(tr);
```

*Figure 7 Scenario setup: IRQ5, IRQ6, IRQ7 active → after delay 10, IRQ1 arrives code*

```
UVM_INFO irq_scoreboard(51) @ 175000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11100000, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=5
UVM_INFO irq_scoreboard(70) @ 185000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=5) == REF(OUT=1,ID=5) ref reg 11000000
UVM_INFO irq_scoreboard(51) @ 185000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11100000, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=6
UVM_INFO irq_scoreboard(70) @ 195000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=6) == REF(OUT=1,ID=6) ref reg 10000000
UVM_INFO irq_scoreboard(51) @ 195000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=00000001, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=0
UVM_INFO irq_scoreboard(70) @ 205000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=0) == REF(OUT=1,ID=0) ref reg 10000000
UVM_INFO irq_scoreboard(51) @ 205000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=00000001, MASK=00000000, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=7
UVM_INFO irq_scoreboard(70) @ 215000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=7) == REF(OUT=1,ID=7) ref reg 00000000
```

*Figure 8 Scenario setup: IRQ5, IRQ6, IRQ7 active → after delay 10, IRQ1 arrives output*

## 4.Masking Test – Ignored IRQ1, Served IRQ2

In this test, IRQ1 and IRQ2 were activated simultaneously while IRQ0 and IRQ1 were masked. As a result, the system ignored IRQ1 due to the mask and proceeded to service IRQ2, confirming that the masking logic correctly prevents masked interrupts from being processed.

```
// --- Transaction 5: mask test (mask IRQ0) ---
tr = irq_seq_item::type_id::create("tr_mask0_test");
tr.irq     = 8'b0000_0110;
tr.mask_in = 8'b0000_0011;
tr.rstn    = 1'b1;
tr.ack     = 1'b1;
start_item(tr);
finish_item(tr);
```

*Figure 9 IRQ1 masked, only IRQ2 serviced code*

```
UVM_INFO irq_scoreboard(51) @ 5000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=00000000, MASK=00000000, RSTn=0, ACK=0, IRQ_OUT=0, IRQ_ID=0
UVM_INFO irq_scoreboard(70) @ 15000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=0,ID=0) == REF(OUT=0,ID=0) ref reg 00000000
UVM_INFO irq_scoreboard(51) @ 15000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=00000110, MASK=00000011, RSTn=1, ACK=1, IRQ_OUT=1, IRQ_ID=2
UVM_INFO irq_scoreboard(70) @ 25000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=1,ID=2) == REF(OUT=1,ID=2) ref reg 00000000
UVM_INFO irq_scoreboard(51) @ 25000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=00000110, MASK=00000011, RSTn=1, ACK=1, IRQ_OUT=0, IRQ_ID=0
UVM_INFO irq_scoreboard(70) @ 35000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=0,ID=0) == REF(OUT=0,ID=0) ref reg 00000000
UVM_INFO irq_scoreboard(51) @ 35000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=00000110, MASK=00000011, RSTn=1, ACK=1, IRQ_OUT=0, IRQ_ID=0
UVM_INFO irq_scoreboard(70) @ 45000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=0,ID=0) == REF(OUT=0,ID=0) ref reg 00000000
```

*Figure 10 IRQ1 masked, only IRQ2 serviced output*

# 5.Testing Full Interrupt Mask Activation

In this test, the interrupt mask (mask_in) was activated for all interrupt lines, preventing the controller from servicing any interrupt requests even if they are active. The purpose of this scenario is to verify that the mask correctly blocks all interrupts.

```
    // --- Mask all 1: all IRQs disabled ---
tr = irq_seq_item::type_id::create("tr_mask_all_one");
tr.irq     = 8'hFF;
tr.mask_in = 8'hFF;
tr.rstn    = 1'b1;
tr.ack     = 1'b1;
start_item(tr);
finish_item(tr);
```

*Figure 11All IRQ lines masked code*

```
UVM_INFO irq_scoreboard(51) @ 5000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=00000000, MASK=00000000, RSTn=0, ACK=0, IRQ_OUT=0, IRQ_ID=0
UVM_INFO irq_scoreboard(70) @ 15000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=0,ID=0) == REF(OUT=0,ID=0) ref reg 00000000
UVM_INFO irq_scoreboard(51) @ 15000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=11111111, RSTn=1, ACK=1, IRQ_OUT=0, IRQ_ID=0
UVM_INFO irq_scoreboard(70) @ 25000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=0,ID=0) == REF(OUT=0,ID=0) ref reg 00000000
UVM_INFO irq_scoreboard(51) @ 25000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=11111111, RSTn=1, ACK=1, IRQ_OUT=0, IRQ_ID=0
UVM_INFO irq_scoreboard(70) @ 35000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=0,ID=0) == REF(OUT=0,ID=0) ref reg 00000000
UVM_INFO irq_scoreboard(51) @ 35000: uvm_test_top.env.sb [IRQ_SCBD] Received TR: IRQ=11111111, MASK=11111111, RSTn=1, ACK=1, IRQ_OUT=0, IRQ_ID=0
UVM_INFO irq_scoreboard(70) @ 45000: uvm_test_top.env.sb [IRQ_SCBD] PASS DUT(OUT=0,ID=0) == REF(OUT=0,ID=0) ref reg 00000000
```

*Figure 12All IRQ lines masked output*

# Conclusion

The all-inclusive verifying based on a UVM environment proved that the Interrupt Controller performs effectively and successfully satisfies the outlined demands. It passed directed and constrained-random tests that demonstrate the capability of handling all interrupt lines properly resolving their priorities, properly performing mask logic, responding correctly to acknowledge signals and supporting Preemption: those interrupts, which are of higher priority, may be serviced during an ongoing service. The performance and stability was considered during the normal and stress operation including edge cases like enabling all the interrupts or masking, and was thus ready to be incorporated in the target processing systems.